

Formatting with f-strings

Christopher Martin - *Bowdoin College* 
c.martin@bowdoin.edu 

Formatting with f-strings

In addition to being able to insert values into strings, f-strings are able to apply special formatting rules to those values too!

This allows them to change the way values are represented
This doesn't change their value, just controls how they appear visually

For example, we can control the way numbers are displayed
Adding a `:` after a value lets you specify additional formatting parameters

```
1 # '.' controls how many points of precision are shown, .2 means 2 decimal places
2 # 'f' means show the value in 'fixed-point' notation
3 # 'e' means show the value in 'scientific' notation
4 print(f"{100} vs {100:.2f} vs {100:.1e}")
```

Width and Justification

Width controls how much space is reserved in the string for a value

Establishes a minimum amount of space reserved for a value, but has no effect on values with a string representation that is longer than the set width

Justification controls where a value is placed within that space

They can be placed on the left `<`, in the middle `^`, or on the right `>`

```
1 first_name = "Chris"
2 last_name = "Martin"
3
4 # Try changing the width and justification! What are the defaults?
5 print(f"|{first_name:<10}|{last_name:<10}|")
```

You can also control the character that fills the extra spaces reserved by a formatted width before the justification!

```
1 first_name = "Chris"
2 last_name = "Martin"
3
4 # Try modifying the width, justification, and fill!
5 print(f"|{first_name:.<10}|{last_name:.<10}|")
6 print(f"|{first_name:=^10}|{last_name:=^10}|")
7 print(f"|{first_name:*>10}|{last_name:*>10}|")
```

```
|Chris.....|Martin....|
|==Chris===|==Martin==|
|*****Chris|*****Martin|
```

Format specification mini-language

“Format specifications” are used within replacement fields contained within a format string to define how individual values are presented (see [Format string syntax](#), [f-strings](#), and [t-strings](#)). They can also be passed directly to the built-in [format\(\)](#) function. Each formattable type may define how the format specification is to be interpreted.

Most built-in types implement the following options for format specifications, although some of the formatting options are only supported by the numeric types.

A general convention is that an empty format specification produces the same result as if you had called [str\(\)](#) on the value. A non-empty format specification typically modifies the result.

The general form of a *standard format specifier* is:

```
format_spec:      [options][width_and_precision][type]
options:          [[fill]align][sign]["z"]["#"]["0"]
fill:             <any character>
align:            "<" | ">" | "=" | "^"
sign:             "+" | "-" | " "
width_and_precision: [width_with_grouping][precision_with_grouping]
width_with_grouping: [width][grouping]
precision_with_grouping: "." [precision][grouping] | "." grouping
width:            digit+
precision:        digit+
grouping:         ",", "_"
type:             "b" | "c" | "d" | "e" | "E" | "f" | "F" | "g"
                  | "G" | "n" | "o" | "s" | "x" | "X" | "%"
```